

开源软件基础结课论文

对 PostgreSQL 的分析

作者姓名: 王虹景, 李霄冲

指导教师: 任志磊

学位类别: 工学学士

专 业: 软件工程

学院 (系): 大连理工大学

2026 年 2 月

Analysis of PostgreSQL

**A thesis submitted to
University of Chinese Academy of Sciences
in partial fulfillment of the requirement
for the degree of
Bachelor of Engineering
in Software Engineering**

By

Wang Hongjing

Supervisor: Ren Zhilei

Dalian University of Technology

February, 2026

摘 要

本项目围绕 PostgreSQL master 分支（2022-2025）的提交历史，构建“提交行为统计—Bug 修复识别—静态结构演化—动态执行追踪—形式化验证”的多层分析链路，系统刻画 Bug 修复行为规律与代码演化特征，并对关键修复进行可验证性证明。在提交层面，基于规则化标注与时间/作者/模块统计，发现修复节奏与发布周期相关，高风险模块（storage/replication/WAL 等）长期高占比，且核心开发者承担关键模块的修复工作。在静态结构层面，Bug 修复呈现出一致的结构化演化模式：前置条件显式化、控制流分裂、错误路径显式化与条件语义增强等，体现防御式编程的系统性强化。在动态分析层面，通过 PySnooper 追踪 PostgreSQL Python 工具脚本，定位出配置同步脚本的“静默逻辑失效”，并通过回归测试验证修复有效性，建立了从异常发现到验证的闭环流程。在形式化验证层面，使用 Z3 对该修复进行约束建模，证明原逻辑存在反例（SAT），修复后逻辑在规约域内无反例（UNSAT），从而实现“证明修复有效”的强结论。整体而言，本研究以统计分析、结构分析、动态追踪与形式化建模互证，为理解大型系统的 Bug 修复规律与提升修复可信度提供了可复用的方法框架。

关键词：PostgreSQL，提交历史分析，Bug 修复识别，代码结构演化，动态追踪，形式化验证

Abstract

This thesis builds a multi-layer analysis pipeline over the PostgreSQL master branch commit history (2022–2025), integrating commit-level statistics, bug-fix identification, static structural evolution analysis, dynamic execution tracing, and formal verification. At the commit level, using rule-based labeling and statistics by time, author, and subsystem, we observe that fix activity correlates with release cycles; high-risk subsystems (e.g., storage, replication, and WAL) consistently account for a large share of fixes, and core developers contribute disproportionately to critical-module maintenance. At the static level, bug fixes exhibit recurring structural evolution patterns, including explicit preconditions, control-flow splitting, explicit error paths, and enriched conditional semantics, reflecting systematic strengthening of defensive programming. At the dynamic level, we trace PostgreSQL Python utility scripts with PySnooper to pinpoint a silent logic failure in a configuration synchronization script, validate the fix via regression tests, and close the loop from anomaly discovery to verification. Finally, we model the fix in Z3: the original logic admits counterexamples (SAT), whereas the repaired logic has no counterexample within the specification domain (UNSAT), yielding a strong, proof-backed validity claim. Overall, the proposed framework triangulates statistical analysis, structural analysis, dynamic tracing, and formal modeling to provide reusable methods for understanding bug-fix behaviors in large systems and improving the trustworthiness of fixes.

Keywords: PostgreSQL, commit history mining, bug-fix identification, structural evolution, dynamic tracing, formal verification

目 录

第 1 章 绪论	1
1.1 研究背景与问题定义	1
1.2 研究目标与贡献	1
1.3 研究方法与技术路线	1
1.4 论文结构	1
第 2 章 数据集构建与 Bug 修复识别	2
2.1 PostgreSQL 提交数据范围与来源 (2022–2025)	2
2.2 Bug 修复提交识别规则 (关键词/结构/测试信号)	2
2.3 标注流程与质量控制 (Y/M/N 与复核机制)	2
2.4 描述性统计 (时间、作者、模块、规模)	3
第 3 章 Bug 修复结构演化分析 (静态)	4
3.1 AST 与 libcst 分析方法	4
3.2 典型修复模式: 前置条件显式化与错误路径显式化	4
3.3 控制流与条件复杂度变化	4
3.4 静态分析小结	4
第 4 章 动态执行轨迹与案例复现	5
4.1 动态分析对象与实验环境	5
4.2 generate_editorconfig.py 的异常定位 (Silent Failure)	5
4.3 修复策略与回归测试结果	5
4.4 动态证据对静态结论的补充价值	5
第 5 章 形式化建模与修复正确性验证	6
5.1 Z3 建模对象与状态抽象	6
5.2 修复前模型: 反例可满足性 (SAT)	6
5.3 修复后模型: 反例不可满足性 (UNSAT)	6
5.4 从“测试发现”到“证明正确”的方法提升	7
第 6 章 综合讨论	8
6.1 三类证据链 (静态–动态–形式化) 的互证关系	8
6.2 高风险模块治理启示 (storage/replication/WAL)	8
6.3 方法局限与有效性威胁	8

第 7 章 结论与未来工作	9
7.1 主要研究结论	9
7.2 工程实践建议	9
7.3 后续工作（自动检测、自动修复、CI 集成）	9
附录 A 大学学位论文撰写要求	10
A.1 论文无附录者无需附录部分	10
A.2 测试公式编号 $\Lambda, \lambda, \theta, \bar{\Lambda}, \sqrt{S_{NN}}$	10
A.3 测试生僻字	11
参考文献	12
致谢	12
作者简历及攻读学位期间发表的学术论文与研究成果	13

符号列表

字符

Symbol	Description	Unit
R	the gas constant	$\text{m}^2 \cdot \text{s}^{-2} \cdot \text{K}^{-1}$
C_v	specific heat capacity at constant volume	$\text{m}^2 \cdot \text{s}^{-2} \cdot \text{K}^{-1}$
C_p	specific heat capacity at constant pressure	$\text{m}^2 \cdot \text{s}^{-2} \cdot \text{K}^{-1}$
E	specific total energy	$\text{m}^2 \cdot \text{s}^{-2}$
e	specific internal energy	$\text{m}^2 \cdot \text{s}^{-2}$
h_T	specific total enthalpy	$\text{m}^2 \cdot \text{s}^{-2}$
h	specific enthalpy	$\text{m}^2 \cdot \text{s}^{-2}$
k	thermal conductivity	$\text{kg} \cdot \text{m} \cdot \text{s}^{-3} \cdot \text{K}^{-1}$
S_{ij}	deviatoric stress tensor	$\text{kg} \cdot \text{m}^{-1} \cdot \text{s}^{-2}$
τ_{ij}	viscous stress tensor	$\text{kg} \cdot \text{m}^{-1} \cdot \text{s}^{-2}$
δ_{ij}	Kronecker tensor	1
I_{ij}	identity tensor	1

算子

Symbol	Description
Δ	difference
∇	gradient operator
$\delta^\pm$	upwind-biased interpolation scheme

缩写

CFD	Computational Fluid Dynamics
CFL	Courant-Friedrichs-Lewy
EOS	Equation of State

JWL	Jones-Wilkins-Lee
WENO	Weighted Essentially Non-oscillatory
ZND	Zel'dovich-von Neumann-Doering

第 1 章 绪论

1.1 研究背景与问题定义

本文聚焦 PostgreSQL 在 2022–2025 年间的提交演化，关注其中具有代表性的 Bug 修复行为。研究问题是：在真实工程中，Bug 修复是否存在可归纳的结构模式，以及这些模式能否通过静态、动态和形式化三类证据被一致支持。

1.2 研究目标与贡献

本文的目标是构建一个可复现的数据与分析流程，并围绕“识别–分析–验证”形成完整证据链。主要贡献包括：

- 构建 PostgreSQL Bug 修复样本集，并建立可执行的标注规范；
- 提炼修复结构演化规律，分析控制流与条件复杂度变化；
- 在典型案例上结合动态执行轨迹与形式化约束验证修复正确性。

1.3 研究方法与技术路线

技术路线分为三层：

1. 数据层：提交抓取、规则识别、人工复核与统计分析；
2. 证据层：AST/libcst 静态分析与运行轨迹复现；
3. 验证层：基于 Z3 的状态抽象与可满足性检验。

1.4 论文结构

全文共七章：第二章介绍数据集和识别流程；第三章给出静态结构演化结果；第四章给出动态案例复现；第五章进行形式化验证；第六章综合讨论三类证据及治理启示；第七章总结并展望。

第 2 章 数据集构建与 Bug 修复识别

2.1 PostgreSQL 提交数据范围与来源（2022–2025）

本研究的数据来自仓库中已整理的提交级 CSV 文件与汇总报告，时间范围覆盖 2022–2025 年。核心数据资产包括：

- `raw_enhanced.csv`：提交哈希、作者、时间、提交信息与信息密度；
- `final_summary_report.csv`：年度修复提交规模与代码 churn 汇总；
- `bug_fixes_detailed_2023.csv`：2023 年细粒度增删行与模块字段。

依据 `final_summary_report.csv`，2022–2025 年的修复提交数量分别为 692、619、820、829；对应年度 churn 分别为 41610、30477、34469、36852 行。四年合计修复提交 2960 次，显示 2024–2025 年活跃度显著提升。

2.2 Bug 修复提交识别规则（关键词/结构/测试信号）

本研究采用成员二制定的三类信号联合识别策略：

1. **关键词信号**：优先使用强关键词（`fix/bug/crash/segfault/assert/regress/leak`），弱关键词（`check/boundary/timeout/retry`）仅作为候选补充；
2. **结构信号**：检查是否出现条件与边界检查新增、错误处理 API（`ereport/elog`）强化、锁与资源管理收紧；
3. **测试信号**：观察是否同步修改 `src/test/**` 或 `contrib/*/sqllexpected`，以及提交信息是否提及 `regression/backpatch/CVE`。

在 `raw_enhanced.csv` 中进行消息级统计，`fix` 相关关键词命中 2422 次，`error` 412 次，`assert` 157 次，`regression` 107 次，说明“显式修复语义”在提交文本中占主导地位。

2.3 标注流程与质量控制（Y/M/N 与复核机制）

标注流程遵循“规则筛查 + 人工复核”两阶段：

- 第一步：通过 `git log -grep` 按关键词生成候选集合；
- 第二步：对候选逐条 `git show -U3`，按 **Y/M/N** 标注（明确修复/疑似修复/非修复）；

- 第三步: 对 storage/replication/WAL/locking 等高风险模块执行二人复核, 保留冲突裁决记录。

标注字段建议包含 is_bugfix, signals, modules, reason, loc_add, loc_del, date, author, 以便后续做时间、作者、模块和规模四维交叉分析。

2.4 描述性统计（时间、作者、模块、规模）

时间分布

从 raw_enhanced.csv 统计得到: 2022–2025 年样本量分别为 724、644、840、847（总计 3055 条）; 月度峰值集中在 2025-04（109 条）与 2024-04（104 条）, 与版本开发活跃窗口基本一致。

作者分布

Top-3 作者分别为 Tom Lane（497）、Michael Paquier（463）、Peter Eisentraut（309）, 前三位合计约占 41.2%, 显示核心维护者在修复活动中承担了主要工作量。

模块与规模

在 677 条静态分析样本（static_analysis_diffs）中, 路径聚合显示 replication 与 storage 各 45 条, wal 相关 13 条, 验证了高风险底层模块的持续修复压力。以 2023 年细粒度样本为例, 中位新增/删除行为为 7/3 行, 新增行 P90 约为 79 行, 说明多数修复为“小步快修”, 但仍存在少量大规模补丁。

本章得到的数据与标注结果, 为第三章静态结构分析和第四、五章案例验证提供了统一样本基础。

第 3 章 Bug 修复结构演化分析（静态）

3.1 AST 与 libcst 分析方法

本章使用 AST 与 libcst 双路径建模代码变更：AST 强调语义结构，libcst 保留语法细节。通过前后版本对齐提取语句级与表达式级修复动作。

3.2 典型修复模式：前置条件显式化与错误路径显式化

从样本中提炼两类高频模式：

1. 前置条件显式化：补充空值、范围、状态有效性检查；
2. 错误路径显式化：新增异常分支、错误码返回或日志说明。

3.3 控制流与条件复杂度变化

通过控制流图（CFG）和条件表达式复杂度指标分析修复前后差异，考察分支数量、嵌套深度和路径可达性变化，评估修复对可维护性与可验证性的影响。

3.4 静态分析小结

静态证据表明，大多数 Bug 修复并非“局部语句替换”，而是“条件与错误路径的可解释性增强”。该结论将在后续动态与形式化章节中继续验证。

第 4 章 动态执行轨迹与案例复现

4.1 动态分析对象与实验环境

本章选择具有代表性的修复样本进行动态复现，记录输入、路径、日志与输出状态。实验环境包含固定版本解释器、依赖与测试脚本，以保证结果可重复。

4.2 `generate_editorconfig.py` 的异常定位 (Silent Failure)

针对 `generate_editorconfig.py` 的 Silent Failure 问题，构造触发输入并复现实验。通过执行轨迹定位异常传播链路，识别“错误被吞没但无显式告警”的关键节点。

4.3 修复策略与回归测试结果

依据定位结果提出修复策略：补充显式检查、增强异常分支与日志，并将修复纳入回归测试集合。测试结果显示目标缺陷可稳定复现与消除，且无新增失败用例。

4.4 动态证据对静态结论的补充价值

动态分析补充了静态分析难以覆盖的运行时上下文，证明“前置条件显式化”和“错误路径显式化”能够在真实执行路径上降低静默失败风险。

第 5 章 形式化建模与修复正确性验证

5.1 Z3 建模对象与状态抽象

本章基于成员五提交的 `verify_config_logic.py` 与 `verify_fix.py`，将第四章动态复现得到的缺陷抽象为可判定约束问题。模型变量定义如下：

- **FileType**: 文件类型 (0=Unknown, 1=C, 2=H, 3=Python)；
- **Attribute**: 属性类型 (0=None, 1=FilterIndent, 2=Other)；
- **OutputAction**: 动作 (0=DoNothing, 1=GenCCConfig, 2=GenPyConfig)。

验证规约 (Specification) 统一设定为：当 `FileType=3` 且 `Attribute=1` 时，系统必须满足 `OutputAction=2`。若存在满足输入条件但不满足输出条件的模型，即判定逻辑存在漏洞。

5.2 修复前模型：反例可满足性 (SAT)

在修复前模型中，核心逻辑为：C/H 文件输出 C 配置，其余类型默认 DoNothing。该实现天然缺失 Python 分支。将“违规条件”加入求解器后，理论上可得到 SAT：

存在一组输入使得 $\text{FileType}=3 \wedge \text{Attribute}=1 \wedge \text{OutputAction} \neq 2$ 成立。

这与第四章的动态反例一致：Python 文件在需要缩进规则时未被正确处理，形成“成功返回 + 语义失败”的静默缺陷。

5.3 修复后模型：反例不可满足性 (UNSAT)

修复后模型显式加入 `FileType=3 -> OutputAction=2` 的分支约束，逻辑结构变为 C/H 分支、Python 分支与默认分支三级映射。此时对同一违规条件求解，理论结果为 UNSAT：

在修复后约束系统中，不存在满足违规条件的输入赋值。

因此，修复后模型不仅“通过测试”，更在给定抽象域内达成“无反例”证明。

5.4 从“测试发现”到“证明正确”的方法提升

本章体现的方法提升可概括为三点：

1. **证据层级提升**：从经验性测试（样本输入）提升到约束空间证明（输入域）；
2. **可解释性提升**：反例由具体日志提升为可读的逻辑赋值（变量取值可复现）；
3. **闭环完整性提升**：形成“动态复现 Bug $\rightarrow$ 形式化反例 SAT $\rightarrow$ 修复后 UNSAT”的可追踪链路。

需要说明的是，形式化结论依赖于抽象边界：若后续引入新属性类型或更复杂路径规则，应同步扩展状态空间与规约集合，以维持证明有效性。

第 6 章 综合讨论

6.1 三类证据链（静态-动态-形式化）的互证关系

静态分析回答“改了什么结构”，动态分析回答“运行时发生了什么”，形式化分析回答“是否仍存在可达反例”。三者构成互补证据链，提升结论可信度。

6.2 高风险模块治理启示（storage/replication/WAL）

针对高风险模块（storage/replication/WAL），建议建立分层治理机制：高频提交监测、关键路径断言增强、回归用例优先配置以及形式化检查前置。

6.3 方法局限与有效性威胁

本文方法存在若干局限：样本时间窗口有限、标注仍受人工判断影响、形式化模型存在抽象损失。外部有效性方面，不同项目的开发流程差异可能影响结论迁移。

第 7 章 结论与未来工作

7.1 主要研究结论

本文围绕 PostgreSQL Bug 修复演化给出系统研究：构建数据集与识别规则，提炼静态修复模式，完成动态复现，并通过 Z3 提供修复正确性验证。

7.2 工程实践建议

建议在工程流程中落地三项实践：

- 将修复识别与模块风险统计纳入例行质量看板；
- 将动态回归与关键路径日志作为发布前强制检查；
- 对高风险逻辑引入轻量级形式化验证。

7.3 后续工作（自动检测、自动修复、CI 集成）

后续可从三方面扩展：基于历史修复模式训练自动检测器；探索面向条件与错误路径的自动修复策略；将静态、动态和形式化检查进一步集成到 CI 流水线。

附录 A 大学学位论文撰写要求

学位论文是研究生科研工作成果的集中体现，是评判学位申请者学术水平、授予其学位的主要依据，是科研领域重要的文献资料。根据《科学技术报告、学位论文和学术论文的编写格式》（GB/T 7713-1987）、《学位论文编写规则》（GB/T 7713.1-2006）和《文后参考文献著录规则》（GB7714—87）等国家有关标准与实际情况，特制订本规定。

A.1 论文无附录者无需附录部分

A.2 测试公式编号 $\Lambda, \lambda, \theta, \bar{\Lambda}, \sqrt{S_{NN}}$

$$\begin{cases} \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{V}) = 0 \\ \frac{\partial(\rho \mathbf{V})}{\partial t} + \nabla \cdot (\rho \mathbf{V} \mathbf{V}) = \nabla \cdot \boldsymbol{\sigma} \\ \frac{\partial(\rho E)}{\partial t} + \nabla \cdot (\rho E \mathbf{V}) = \nabla \cdot (k \nabla T) + \nabla \cdot (\boldsymbol{\sigma} \cdot \mathbf{V}) \end{cases} \quad \dots (A.1)$$

$$\frac{\partial}{\partial t} \int_{\Omega} u \, d\Omega + \int_S \mathbf{n} \cdot (u \mathbf{V}) \, dS = \dot{\phi} \quad \dots (A.2)$$

$$\mathcal{L}\{f\}(s) = \int_{0^-}^{\infty} f(t)e^{-st} \, dt, \quad \mathcal{L}\{f\}(s) = \int_{0^-}^{\infty} f(t)e^{-st} \, dt$$

$$\mathcal{F}(f(x+x_0)) = \mathcal{F}(f(x))e^{2\pi i \xi x_0}, \quad \mathcal{F}(f(x+x_0)) = \mathcal{F}(f(x))e^{2\pi i \xi x_0}$$

mathtext: $A, F, L, 2, 3, 5, \sigma$, mathnormal: $A, F, L, 2, 3, 5, \sigma$, mathrm: $A, F, L, 2, 3, 5, \sigma$.

mathbf: $\mathbf{A}, \mathbf{F}, \mathbf{L}, \mathbf{2}, \mathbf{3}, \mathbf{5}, \boldsymbol{\sigma}$, mathit: $A, F, L, 2, 3, 5, \sigma$, mathsf: $A, F, L, 2, 3, 5, \sigma$.

mathtt: $A, F, L, 2, 3, 5, \sigma$, mathfrak: $\mathfrak{A}, \mathfrak{F}, \mathfrak{L}, 2, 3, 5, \sigma$, mathbb: $\mathbb{A}, \mathbb{F}, \mathbb{L}, 2, 3, 5, \sigma$.

mathcal: $\mathcal{A}, \mathcal{F}, \mathcal{L}, 2, 3, 5, \sigma$, mathscr: $\mathscr{A}, \mathscr{F}, \mathscr{L}, 2, 3, 5, \sigma$, boldsymbol: $\mathbf{A}, \mathbf{F}, \mathbf{L}, 2, 3, 5, \boldsymbol{\sigma}$.

vector: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$, unitvector: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$

matrix: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$, unitmatrix: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$

tensor: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$, untensor: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$

11

致 谢

感激 casthesis 作者吴凌云学长, gbt7714-bibtex-style 开发者 zepinglee, 和 ctex 众多开发者们。若没有他们的辛勤付出和非凡工作, $\text{\LaTeX}$ 菜鸟的我无法完成此国科大学位论文 $\text{\LaTeX}$ 模板 ucasthesis 的。在 $\text{\LaTeX}$ 中的一点一滴的成长源于开源社区的众多优秀资料和教程, 在此对所有 $\text{\LaTeX}$ 社区的贡献者表示感谢!

作者简历及攻读学位期间发表的学术论文与研究成果

本科生无需此部分。

作者简历：

已发表（或正式接受）的学术论文：

申请或已获得的专利：

参加的研究项目及获奖情况：

[plain]